Implementation of the sectional method

Gregor Neuber

Universität Stuttgart

Institut für Technische Verbrennung

Population Balance Equation

The Population Balance Equation (PBE) describes the evolution of the particle number density N in time, space and phase space:

$$\frac{\partial N(d_p,\overline{x},t)}{\partial t} + \nabla\left[\overline{u}\cdot N(d_p,\overline{x},t)\right] + \frac{\partial}{\partial d_p}\left[G(d_p,\overline{Y})\cdot N(d_p,\overline{x},t)\right] = D_p\Delta^2 N(d_p,\overline{x},t) + \dot{\omega}_N(\overline{Y},N)$$

Condensation (drift term) **Nucleation (source term)**

In the sectional method the actual continuous PBE is discretised into sections which symbolizes the particle size distribution here. This discretisation generates a high-dimensional problem which is somehow similar to the high-dimensional system of species composition in combustion approaches.

This talk presents the implementation of the sectional method into the already existing *mmcFoam* framework.

Sectional method

Since the introduced sections cover an encapsulated physical process all necessary functions will be put into their own Pope particle and cloud classes. These classes will be called *sectionalPopeParticle* and *sectionalPopeCloud*, respectively.

The newly introduced sections need to be mixed and the sub-models of the sectional class need access to the thermo-physical properties. Therefore the class will be put between the *thermoPopeCloud* and *mixingPopeCloud* level.

Originally it was tried to remain the pure gas phase solver *mmcFoam* unchanged and to introduce a new solver, e.g. mmcPbeFoam, which instantiate a cloud which inherited from the sectionPopeCloud. This was not possible now and must be done later.

The new classes where created by ‘cherry picking’ from the already existing classes. The sections are introduced via the OpenFOAM internal *scalarField* class.

Only the key aspects of introducing a new cloud level and not the complete code additions are shown here.

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/Templates/

- ItoPopeParticle
- MixingPopeParticle
- ReactingPopeParticle
- SectionalPopeParticle ← New class template is introduced. Copied from MixingPopeParticle template.
- ThermoPopeParticle

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/Templates/SectionalPopeParticle

- SectionalPopeParticle.C
- SectionalPopeParticle.H
- SectionalPopeParticleI.H
- SectionalPopeParticleIO.C
- SectionalPopeParticleTrackingDataI.H

Renaming of copied files.

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/Templates/SectionalPopeParticle

```
Class
    Foam::SectionalPopeParticle

Description
    particle which introduces the sectional method

SourceFiles
    SectionalPopeParticleI.H
    SectionalPopeParticle.C
    SectionalPopeParticleIO.C

\*---------------------------------------------------------------------------*/

#ifndef SectionalPopeParticle_H
#define SectionalPopeParticle_H

#include "particle.H"


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

template<class ParticleType>
class SectionalPopeParticle;

template<class ParticleType>
Ostream& operator<<
(
    Ostream&,
    const SectionalPopeParticle<ParticleType>&
);

/*---------------------------------------------------------------------------*\
                        Class SectionalPopeParticle Declaration
\*---------------------------------------------------------------------------*/
```

Replace ‘MixingPopeParticle’ with ‘SectionalPopeParticle’ in all files for the sectional particle class definition.

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/derived/

- basicItoPopeParticle
- basicMixingPopeParticle
- basicReactingPopeParticle
- basicSectionalPopeParticle ← To change the inheritances introduce new inheritance level for the sectional Pope particle.
- basicThermoPopeParticle

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/derived/basicSectionalPopeParticle/basicSectionalPopeParticle.H

```
#ifndef basicSectionalPopeParticle_H
#define basicSectionalPopeParticle_H

#include "contiguous.H"
#include "particle.H"
#include "ItoPopeParticle.H"
#include "ThermoPopeParticle.H"
#include "SectionalPopeParticle.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

namespace Foam
{
    typedef
    SectionalPopeParticle
    <
        ThermoPopeParticle
        <
            ItoPopeParticle
            <
                particle
            >
        >
    >
    basicSectionalPopeParticle;

    template<>
    inline bool contiguous<basicSectionalPopeParticle>()
    {
        return true;
    }
}
```

To change the inheritances introduce new inheritance level for the sectional Pope particle.

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/derived/basicMixingPopeParticle/basicMixingPopeParticle.H

```
#include "contiguous.H"
#include "particle.H"
#include "ItoPopeParticle.H"
#include "ThermoPopeParticle.H"
#include "SectionalPopeParticle.H"
#include "MixingPopeParticle.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

namespace Foam
{
    typedef
    MixingPopeParticle
    <
        SectionalPopeParticle
        <
            ThermoPopeParticle
            <
                ItoPopeParticle
                <
                    particle
                >
            >
        >
    >
    basicMixingPopeParticle;

    template<>
    inline bool contiguous<basicMixingPopeParticle>()
    {
        return true;
    }
}
```

Change the inheritance line for the mixing level (reacting level not shown here).

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/derived/basicSectionalPopeParticle/makeBasicSectionalPopeParticleSubmodels.C

Create basicSectionalPopeParticle sub-models.

```
#include "basicSectionalPopeCloud.H"

// Kinematic
#include "makeThermoParticleInflowBoundaryModels.H"

// Thermodynamic
#include "makeThermoPopeParticleRadiationModels.H"
#include "makeThermoPopeParticleCompositionModels.H"

// Sectional
#include "makeSectionalPopeParticleSynthesisModels.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    // Kinematic sub-models
    makeThermoParticleInflowBoundaryModels(basicSectionalPopeCloud);

    // Thermo sub-models
    makeThermoPopeParticleCompositionModels(basicSectionalPopeCloud);
    makeThermoPopeParticleCloudRadiationModels(basicSectionalPopeCloud);

    // Sectional sub-models
    makeSectionalPopeParticleSynthesisModels(basicSectionalPopeCloud);
}
```

OF Modifications: particle level

src/lagrangian/mmc/popeParticles/include/makeSectionalPopeParticleSynthesisModels.H

```
#ifndef makeSectionalPopeParticleSynthesisModels_H

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "HomogeneousNucleationCondensation.H"
#include "NoSynthesis.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#define makeSectionalPopeParticleSynthesisModels(CloudType)                    \
                                                                                \
    makeSynthesisModel(CloudType);                                              \
    makeSynthesisModelType(NoSynthesis, CloudType);                             \
    makeSynthesisModelType(HomogeneousNucleationCondensation, CloudType);

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

Add the synthesis sub-models on sectional Pope particle level.

OF Modifications: cloud level

src/lagrangian/mmc/clouds/Templates

- ItoPopeCloud
- MixingPopeCloud
- ReactingPopeCloud
- SectionalPopeCloud ← New class template is introduced. Copied from MixingPopeCloud template.
- ThermoPopeCloud

OF Modifications: cloud level

src/lagrangian/mmc/clouds/Templates/SectionalPopeCloud

- SectionalPopeCloud.C
- SectionalPopeCloud.H
- SectionalPopeCloudI.H

Replace ‘MixingPopeCloud’ with ‘SectionalPopeCloud’ in all files for the sectional particle class definition.

Example:

```
namespace Foam
{
    // Forward declaration of classes

template<class CloudType>
class SynthesisModel;

/*---------------------------------------------------------------------------*\
Class SectionalPopeCloud Declaration
\*---------------------------------------------------------------------------*/

template<class CloudType>
class SectionalPopeCloud
:
public CloudType,
public sectionalPopeCloud
{
    public:

    // Public typedefs

        //- Type of cloud this cloud was instantiated for
        typedef CloudType cloudType;

        //- Type of parcel the cloud was instantiated for
        typedef typename CloudType::particleType particleType;

        //- Convenience typedef for this cloud type
        typedef SectionalPopeCloud<CloudType> sectionalPopeCloudType;
```

OF Modifications: cloud level

src/lagrangian/mmc/clouds/baseClasses/sectionalPopeCloud/

- sectionalPopeCloud.C
- sectionalPopeCloud.H

Copy mixingPopeCloud base class and do some renaming.

Example: sectionalPopeCloud.C

```
#include "sectionalPopeCloud.H"

// * * * * * * * * * * * * * * * Static Data Members * * * * * * * * * * * * * * * //

namespace Foam
{
    defineTypeNameAndDebug(sectionalPopeCloud, 0);
}


// * * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * * * * //

Foam::sectionalPopeCloud::sectionalPopeCloud()
{}


// * * * * * * * * * * * * * * * * * *  Destructors  * * * * * * * * * * * * * * * * * //

Foam::sectionalPopeCloud::~sectionalPopeCloud()
{}
```

OF Modifications: cloud level

src/lagrangian/mmc/clouds/derived/

- basicItoPopeCloud
- basicMixingPopeCloud
- basicReactingPopeCloud
- basicSectionalPopeCloud ←
- basicThermoPopeCloud

To change the inheritances introduce new inheritance level for the sectional Pope cloud.

OF Modifications: cloud level

src/lagrangian/mmc/clouds/derived/basicSectionalPopeCloud/basicSectionalPopeCloud.H

```
#ifndef basicSectionalPopeCloud_H
#define basicSectionalPopeCloud_H

#include "ItoPopeCloud.H"
#include "ThermoPopeCloud.H"
#include "SectionalPopeCloud.H"
#include "basicSectionalPopeParticle.H"

// * * * * * * * * * * * * * * * * * * * * * * * * *

namespace Foam
{
    typedef SectionalPopeCloud
    <
        ThermoPopeCloud
        <
            ItoPopeCloud
            <
                Cloud
                <
                    basicSectionalPopeParticle
                >
            >
        >
    >
    basicSectionalPopeCloud;
}
```

To change the inheritances introduce new inheritance level for the sectional Pope cloud.

OF Modifications: cloud level

src/lagrangian/mmc/clouds/derived/basicMixingPopeCloud/basicMixingPopeCloud.H

```
#ifndef basicMixingPopeCloud_H
#define basicMixingPopeCloud_H

#include "ItoPopeCloud.H"
#include "ThermoPopeCloud.H"
#include "SectionalPopeCloud.H"
#include "MixingPopeCloud.H"
#include "basicMixingPopeParticle.H"

// * * * * * * * * * * * * * * * * * * * * * * * * *

namespace Foam
{
    typedef MixingPopeCloud
    <
        SectionalPopeCloud
        <
            ThermoPopeCloud
            <
                ItoPopeCloud
                <
                    Cloud
                    <
                        basicMixingPopeParticle
                    >
                >
            >
        >
    >
    basicMixingPopeCloud;
}
```

Change the inheritance line for the mixing level (reacting level not shown here).

OF Modifications: cloud level

src/lagrangian/mmc/clouds/Templates/SectionalPopeCloud/SectionalPopeCloud.C

In function *evolve*

```
template<class CloudType>
void Foam::SectionalPopeCloud<CloudType>::evolve()
{
    //- Construct a particleType::trackingData object
    typename particleType::template TrackingData<SectionalPopeCloud<CloudType> > td(*this);

    //- Particle injection at inflow boundaries
    this->inflowBoundary().inflow(td);

    this->solve(td);

    synthesis().synthesize();
}
```

← Add synthesis.

OF Modifications: cloud level

src/lagrangian/mmc/clouds/Templates/MixingPopeCloud/MixingPopeCloud.C

In function *evolve*

```
template<class CloudType>
void Foam::MixingPopeCloud<CloudType>::evolve()
{
    typename particleType::template TrackingData<MixingPopeCloud<CloudType> >
        td(*this);

    //- Particle injection at inflow boundaries
    this->inflowBoundary().inflow(td);
    //    this->inflowBoundary().inflow();

    this->solve(td);

    this->synthesis().synthesize();

    mixing().Smix();
}
```

← Add synthesis on mixing (and reacting) level.

For later revisions it might be good to transform the evolve functions to virtual functions (like the Eulerian statistics functions). Then one is not running into trouble for different solver inheritances.

OF Modifications: sub-model level

src/lagrangian/mmc/submodels/

- Ito/InflowBoundaryModel
- Mixing/MixingModel
- Reacting/ReactionModel
- Sectional/SynthesisModel
- Thermodynamic
- SubModelBase.C
- SubModelBase.H

Copy sub-model directory for the sectional method. Here called *SynthesisModel*.

OF Modifications: sub-model level

src/lagrangian/mmc/submodels/Sectional/SynthesisModel/

- HomogeneousNucleationCondensation
- NoSynthesis
- SynthesisModel

The HomogeneousNucleationCondensation sub-model is the first for the sectional method level.

OF Modifications: sub-model level

src/lagrangian/mmc/submodels/Mixing/MixingModel/ParticleInteractionModels/mixParticleModel/mixParticleModel.H

```
template<class CloudType>
class mixParticleModel
:
    public CloudMixingModel<CloudType>
{
    protected:
        scalar fLow_;
        scalar fHigh_;

    //- Mixing and turbulence field parameters
        const volScalarField& f_;
        const volScalarField& DEff_;

    //- List of all particles
        DynamicList<mixParticle> particleList_;

        List<label> listStart_;

    //- Indexes in mixParticleModel
        label numSp_;
        label nSections_;
        const label nI_;
        const label nx_;
        const label ny_;
        const label nz_;
        const label nwt_;
        const label nf_;
        const label nrand_;
        const label nMagSqr_;
        const label nDEff_;
        const label nmf_;
        const label ndx_;
        const label ndf_;
        const label nh_;
        const label nYs_;
        label nYe_;
        label nSs_;
        label nSe_;

    public:
    // Constructors
```

```
        //- Apply mix
        virtual void Smix();

        void SmixNew
        (
            scalar fLow,
            scalar fHigh,
            bool mixOnMaster
        );


        virtual void SmixList(DynamicList<mixParticle> & particleList) {}


    // Inline

        inline label numSp()
        {
            return numSp_;
        };

        inline label nSections()
        {
            return nSections_;
        };
};

} // End namespace Foam
```

Since the sections must be mixed they have to be considered in the mixing sub-model too.

OF Modifications: sub-model level

src/lagrangian/mmc/submodels/Mixing/MixingModel/ParticleInteractionModels/mixParticleModel/mixParticleModel.H

In the class (copy) constructor *mixParticleModel*

```
    nf_(nwt_ + 1),
    nrand_(nf_ + 1),
    nMagSqr_(nrand_ +1),
    nDEff_(nMagSqr_ + 1),
    nmf_(nDEff_ + 1),
    ndx_(nmf_ + 1),
    ndf_(ndx_ + 1),
    nh_(ndf_ + 1),
    nYs_(nh_ + 1),
    nYe_(nYs_ + 1),
    nSs_(nYe_ + 1),
    nSe_(0)
{}
```

OF Modifications: sub-model level

src/lagrangian/mmc/submodels/Mixing/MixingModel/ParticleInteractionModels/mixParticleModel/mixParticleModel.H

In the member function *buildParticleList*

```
template <class CloudType>
void Foam::mixParticleModel<CloudType>::buildParticleList
(
    scalar fLow,
    scalar fHigh
)
{
    volScalarField magSqr_f = magSqr(fvc::grad(this->f_));

    interpolationCellPoint<scalar> magSqr_intp_(magSqr_f);

    interpolationCellPoint<scalar> DEff_intp_(this->DEff_);

    particleList_.clear();

    //- mixParticle list length is number of non-species scalars + number of species
    numSp_ = this->owner().composition().componentNames().size();
    nYe_ = nYs_ + numSp_ - 1;

    nSections_ = this->owner().nSections();
    nSs_ = nYe_ + 1;
    nSe_ = nSs_ + nSections_ - 1;

    label numP = nSe_ + 1;

    label I = 0;

    StochasticLib1 rand(time(0));

    forAllIter(typename CloudType, this->owner(), iter)
    {
        if (iter().f() < fLow || iter().f() > fHigh)
            continue;

        mixParticle p(numP);
```

•
•
•

```
        p[nh_] = iter().hA();

        //- Species
        label n = nYs_;
        forAll(iter().Y(), j)
        {
            p[n] = iter().Y()[j];

            n++;
        }

        //- Sections
        label m = nSs_;
        forAll(iter().sections(), j)
        {
            p[m] = iter().sections()[j];

            m++;
        }

        particleList_.append(p);

        I++;
    }
}
```

OF Modifications: cloud level

src/lagrangian/mmc/Make/files

```
PARTICLES=popeParticles
BASEPARTICLES=$(PARTICLES)/baseClasses
DERIVEDPARTICLES=$(PARTICLES)/derived

CLOUDS=clouds
BASECLOUDS=$(CLOUDS)/baseClasses
DERIVEDCLOUDS=$(CLOUDS)/derived

/* Cloud base classes */
$(BASECLOUDS)/itoPopeCloud/itoPopeCloud.C
$(BASECLOUDS)/thermoPopeCloud/thermoPopeCloud.C
$(BASECLOUDS)/sectionalPopeCloud/sectionalPopeCloud.C
$(BASECLOUDS)/mixingPopeCloud/mixingPopeCloud.C
$(BASECLOUDS)/reactingPopeCloud/reactingPopeCloud.C
```

Adaptation of new cloud class into the makefile.

```
/* Ito Particle sub-models*/
ITOPOPEPARTICLE=$(DERIVEDPARTICLES)/basicItoPopeParticle
$(ITOPOPEPARTICLE)/defineBasicItoPopeParticle.C
$(ITOPOPEPARTICLE)/makeBasicItoPopeParticleSubmodels.C

/* Thermo Particle sub-models*/
THERMOPOPEPARTICLE=$(DERIVEDPARTICLES)/basicThermoPopeParticle
$(THERMOPOPEPARTICLE)/defineBasicThermoPopeParticle.C
$(THERMOPOPEPARTICLE)/makeBasicThermoPopeParticleSubmodels.C

/* Sectional Particle sub-models*/
SECTIONALPOPEPARTICLE=$(DERIVEDPARTICLES)/basicSectionalPopeParticle
$(SECTIONALPOPEPARTICLE)/defineBasicSectionalPopeParticle.C
$(SECTIONALPOPEPARTICLE)/makeBasicSectionalPopeParticleSubmodels.C

/* Mixing Particle sub-models*/
MIXINGPOPEPARTICLE=$(DERIVEDPARTICLES)/basicMixingPopeParticle
$(MIXINGPOPEPARTICLE)/defineBasicMixingPopeParticle.C
$(MIXINGPOPEPARTICLE)/makeBasicMixingPopeParticleSubmodels.C

/* Reacting Particle sub-models*/
REACTINGPOPEPARTICLE=$(DERIVEDPARTICLES)/basicReactingPopeParticle
$(REACTINGPOPEPARTICLE)/defineBasicReactingPopeParticle.C
$(REACTINGPOPEPARTICLE)/makeBasicReactingPopeParticleSubmodels.C

/* Additional helper classes */
clouds/releaseCloud/releaseCloud.C
clouds/Templates/ItoPopeCloud/popeCloudSolution/popeCloudSolution.C
```

Adaptation of new sectional sub-models into the makefile.

```
LIB = $(FOAM_USER_LIBBIN)/libppParticle
```

Tutorial for sectional method

tutorials/sectionalMethod/HomogeneousNucleationCondensation/DBP/

Numerical setup to model homogeneous nucleation and condensation processes which were measured in the experimental setup of Lesniewski and Friedlander (1998):

Thomas K. Lesniewski and Sheldon K. Friedlander, Particle nucleation and growth in a free turbulent jet, *Proc. R. Soc. Lond.* A (1998) **454**, 2477 - 2504.

A turbulent hot DBP laden nitrogen jet mixes with cold air. The mixture gets supersaturated and this consequently leads to nucleation and surface growth.

Tutorial for sectional method: setup

tutorials/sectionalMethod/HomogeneousNucleationCondensation/DBP/constant/cloudProperties

```
thermophysicalCoupling
{
    enabled          false;

    // List of species for which equivalent mass fraction solutions are obtained on the FV grid
    N2;
    O2;
    C16H22O4;

    primarySpecies   C16H22O4;

    tauRelax         10;
    tauUnits         timestep;
}

sectionalMethodProperties
{
    nSections 30;
}

eulerianStatistics
{
    enabled          false;
    axisymmetric     true;
    symmetryPlane    zx;

    z;
    f;
    T;
    wt;
```

Set size of *scalarField* sections.

Tutorial for sectional method: setup

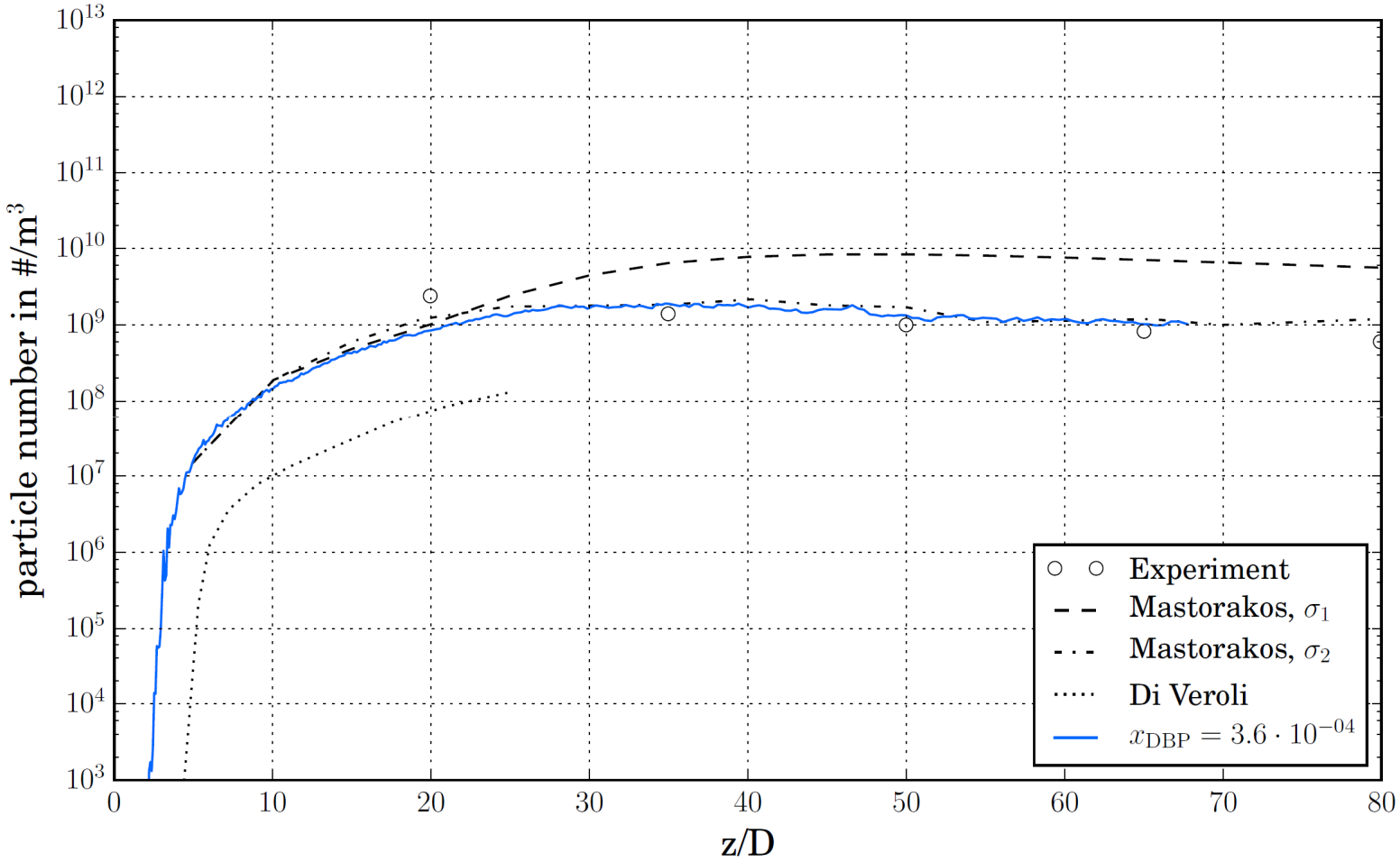
tutorials/sectionalMethod/HomogeneousNucleationCondensation/DBP/constant/cloudProperties

```
reactionModel                 none;

synthesisModel                homogeneousNucleationCondensation;

homogeneousNucleationCondensationCoeffs
{
    nucleatingSpecies         C16H22O4;

    liquidDensityModelCoeffs
    {
        //- density [kg/m^3] = m*T + n
        m     -0.8;
        n     1200;
    }

    surfaceTensionModelCoeffs
    {
        //- sigma [kg/s^2] = m*T + n
        m    -0.0002;
        n     0.1100;
    }

    saturationPressureModelCoeffs
    {
        //- Psat [Pa] = 10^(a*T^2 + b*T + c);
        a    -0.00012;
        b     0.12;
        c    -30;
    }

    nucleationRateModel       veroli;
    condensationRateModel     FuchsSutugin;

    sizeZero                  2.0e-09;
    growthFactor              1.2;
}
```

Set properties for sub-model.

Tutorial for sectional method: results

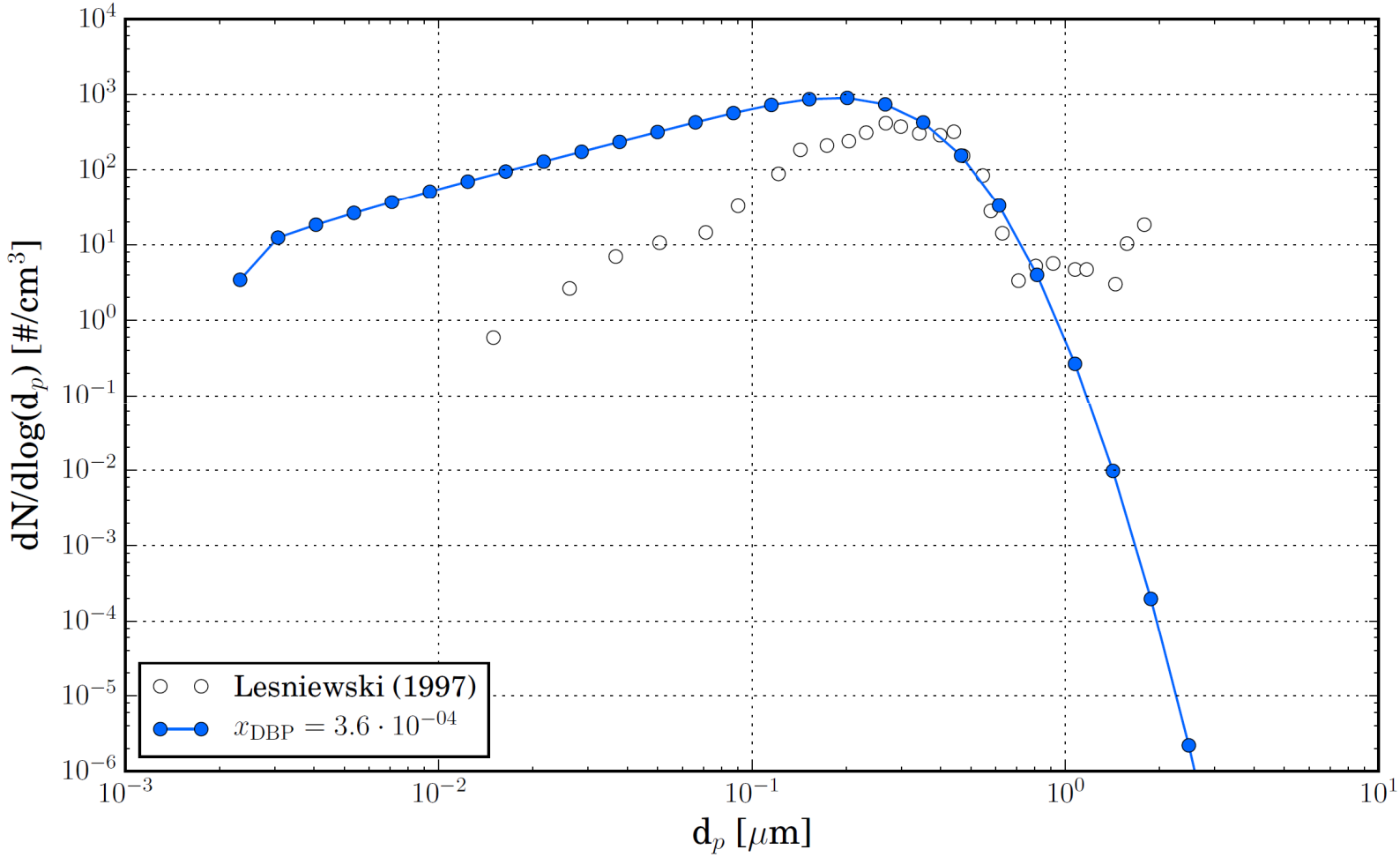
tutorials/sectionalMethod/HomogeneousNucleationCondensation/DBP/



Particle number density (zeroth moment) along centerline for the base case with $x_{DBP} = 3.6 \cdot 10^{-4}$

Tutorial for sectional method: results

tutorials/sectionalMethod/HomogeneousNucleationCondensation/DBP/



mean particle size distribution (PSD) at z/D = 20